Waterfall Streams

AlgoExpert · Very Hard · Arrays

The problem

- Water poured at the top of `source` column falls through a grid of 0 (air) and 1 (block).
- On hitting a block it splits half left, half right, flowing sideways until it can fall (or is lost at a wall).
- Return percentages reaching each bottom-row column.
- Row-by-row simulation.

Intuition

- Simulate row by row: water above a block splits left and right until it finds a gap to fall through.

Brute force vs optimal

- Simulate each droplet → slow
- Row-by-row simulation of amounts → $O(w^2 \cdot h)$

Approach

- 1. Carry percentages per column
- 2. On a block, split half left and half right
- 3. Each half moves until a gap or a wall
- 4. Return the bottom row

Worked example

```
water enters column 3 at the top

falls until it hits a block (1)
```

Water falls straight down

Worked example

```
on a block: split 50/50 left and right

each half slides until it finds a gap below
```

Splits on a block

Worked example

bottom row collects percentages

e.g. [0, 0, 0, 25, 25, 0, 0]

Percentages at the bottom

Java solution

```java
public class WaterfallStreams {
    public static double[] waterfallStreams(double[][] array, int source) {
        int w = array[0].length;
        double[] row = new double[w];
        row[source] = -1; // negative = water amount, to distinguish from blocks
        for (int r = 1; r < array.length; r++) {
            double[] cur = array[r].clone();
            for (int c = 0; c < w; c++) {
                double water = row[c];
                if (water >= 0) continue;
                if (cur[c] != 1.0) { cur[c] += water; continue; }
                double split = water / 2;
                for (int k = c - 1; k >= 0; k--) {
                    if (row[k] == 1.0) break;
                    if (cur[k] != 1.0) { cur[k] += split; break; }
                }
                for (int k = c + 1; k < w; k++) {
                    if (row[k] == 1.0) break;
                    if (cur[k] != 1.0) { cur[k] += split; break; }
                }
            }
            row = cur;
        }
        double[] out = new double[w];
        for (int c = 0; c < w; c++) out[c] = row[c] < 0 ? -row[c] * 100 : 0;
        return out;
    }
```

Complexity

- Time: O(w^2 * h)
- Space: O(w)

Edge cases & pitfalls

- Water trapped by walls is lost
- Use negatives or a separate array to distinguish water from blocks
- Floating-point percentages

Interview tips & follow-ups

- Draw a small example

Recap

- Simulate row by row: water above a block splits left and right until it finds a gap to fall through.
- Complexity: O(w^2 * h)